

Virtual Memory:

A method where secondary memory acts like main memory. Addresses used by programs are automatically converted to physical addresses. The size depends on the computer's addressing system and secondary memory.

Virtual Address:

An address used in virtual memory, allowing a location to act as if it is in main memory.

Virtual Address Space:

The total virtual memory assigned to a process.

Address Space:

The range of memory addresses a process can use.

Real Address:

The actual location of data in main memory.

Two fundamental characteristics of memory management:**(Logical to Physical Address Translation)**

1. All memory references are **logical addresses** translated into **physical addresses** at runtime.

(Non-Contiguous Memory Allocation)

2. A process can be divided into multiple non-contiguous parts in main memory and only needed parts are loaded into memory , executed in this way.

Summary of Execution of a Process:**1. Loading into Memory:**

- The operating system loads parts of the program (resident set) into main memory.

2. Interrupt Handling:

- An interrupt occurs if a required address is not in main memory.
- The process is placed in a blocking state while the missing part is fetched.

3. Disk I/O:

- The operating system issues a disk I/O request to bring the required piece into memory.
- While disk I/O is happening, another process runs.
- Once disk I/O is complete, an interrupt places the affected process in the ready state.

Implications (Short Summary):

1. **More processes fit in memory:** Only parts of a process are loaded, allowing more processes to be in a ready state, improving efficiency.
2. **Processes can exceed memory size:** Virtual memory dynamically loads needed parts, eliminating memory size limitations for programmers.

Real and Virtual Memory (Summary):

- **Real Memory:** The actual main memory (RAM) where processes are executed.
- **Virtual Memory:** Memory on the disk that appears as extra memory to users and allows larger processes to run.

Summary of Table 8.2: Characteristics of Paging and Segmentation

1. Simple Paging:

- Memory is divided into **fixed-size frames**, and processes are divided into **pages** of the same size.
- Causes **internal fragmentation** but avoids **external fragmentation**.
- A **page table** is required to track page locations.

2. Virtual Memory Paging:

- Not all pages need to be in main memory; pages are loaded as needed.
- Helps manage larger processes using disk storage.

3. Simple Segmentation:

- Memory is divided into **variable-sized segments** based on logical divisions of a process (e.g., code, data).
- No internal fragmentation, but **external fragmentation** can occur.
- A **segment table** is required to track segment locations.

4. Virtual Memory Segmentation:

- Segments are loaded into memory as needed.
- Allows large processes to run by storing segments on disk.
- May require writing segments out to disk when new segments are loaded.

Thrashing :

- Thrashing occurs when a system spends **most of its time swapping process pieces** in and out of memory instead of executing instructions. This reduces overall performance.
- To prevent thrashing, the operating system predicts which process pieces are **least likely to be used soon**, based on recent usage history, and avoids unnecessary swapping.

Principle of Locality :

- Programs and data are often accessed in **clusters**.(küme halinde)
- Only a **small part** of a process is needed at a time.
- The operating system can **predict** which parts will be needed soon.
- Helps in **preventing thrashing** by reducing unnecessary swaps.

Support Needed for Virtual Memory

- **Hardware** must support paging and segmentation.
- **Operating system** needs software to manage the movement of pages and/or segments between secondary memory and main memory.

Paging

- Virtual memory is usually used with **paging systems**.
- Paging for virtual memory was first used in the Atlas computer.
- Each process has a **page table**.
 - Page table entries (PTE) store the frame number for each page.
 - A page table is essential for virtual memory systems using paging.

Inverted Page Table

- A hash value is created from the page number of a virtual address, pointing to the inverted page table.
- The table size depends on physical memory, not on the number of processes or virtual pages.
- It's called "inverted" because it uses frame numbers, not virtual page numbers, for indexing.

Inverted Page Table



- **Page number** : This is the page number part of the virtual address
- **Control bits**: Contains flags, protection, and locking info.
- **Process identifier**: The process that owns this page
- **Chain pointer**: Points to the next entry in case of hash collisions.

Translation Lookaside Buffer (TLB) Summary:

- Each virtual memory reference involves **two physical memory accesses**:
 1. To fetch the page table entry.
 2. To fetch the actual data.
- To reduce the increased memory access time, a **Translation Lookaside Buffer (TLB)** is used.
 - TLB is a high-speed cache storing recently used page table entries.
 - It works similarly to a memory cache, improving performance by minimizing direct memory access.
- The TLB only contains some of the page table entries so we cannot simply index into the TLB based on page number

Page Size

Page size matters in memory. Small pages reduce wasted space inside memory blocks but increases size of page tables. Big pages move data faster. The size depends on how big the memory and program are.

Segmentation

Segmentation allows the programmer to view memory as consisting of multiple address spaces or segments. It offers several benefits:

- Simplifies handling of **large and growing data structures**.
- Allows **independent modification** and recompilation of programs.
- Supports **sharing of data** between processes.
- Raise **memory protection** for processes.



Combined Paging and Segmentation



In combined paging and segmentation, memory is divided into segments based on the program's parts, like code or data. Each segment is then split into fixed-size pages matching memory frames.

- **Segmentation**: Programmer can see the segments.
- **Paging**: Programmer cannot see the pages; it's handled by the system.

Protection and Sharing in Segmentation

- **Segmentation helps with protection and sharing policies** by dividing memory into segments.
- **Each segment entry has a base address and length**, which controls access and prevents unauthorized memory usage.
- **Segments can be shared** among multiple processes by allowing them to reference the same segment.

The memory management design of an operating system depends on three fundamental areas of decisions:

1. Whether to use virtual memory or not.
2. Whether to apply paging, segmentation, or both.
3. Which algorithms to use for memory management.



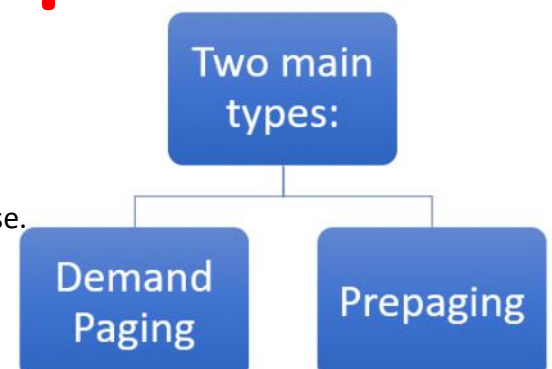
Fetch Policy



Fetch Policy decides when a page should be loaded into memory.

There are **two main types**:

1. **Demand Paging**: Loads a page only when it is needed.
2. **Prepaging**: Loads multiple pages at once, anticipating future use.



Demand Paging

- Pages are loaded into memory only **when needed**.
- At the start of a process, many page faults occur.
- Over time, **because the** program keeps using the same parts of memory again and again(**principle of locality**) , it won't need to load new pages as often, so there will be **fewer errors (page faults)**.



Preparing

- Brings extra pages into memory, not just the one needed.
- Tries to make loading faster by bringing nearby pages together.
- Works well if those extra pages are used; otherwise, it wastes memory.
- It's not the same as swapping.



Placement Policy

- Decides where a part of a process should go in main memory.
- Important in segmentation systems.
- Paging and combined paging/segmentation work efficiently due to hardware.
- Automatic placement is useful for NUMA systems.

Replacement Policy

- Selects which page in memory to remove when a new page needs space.
- The goal is to remove the least-used page.
- Complex policies need more hardware and software resources.

Frame Locking

- Prevents locked frames from being replaced.
- OS kernel and key control data are stored in locked frames.
- Critical I/O buffers can be locked.
- Locking is done by using a lock bit for each frame.